评分人	X

1. 已知带头结点的非空单链表 L 的头指针为 h，指针 p 指向 L 中间的一个链表结点（不是第一个和最后一个结点）。q=p->next, p->next=q->next, q->next=h->next, h->next=q。这段代码的功能是（ D ）。

A. 把 q 指向的结点插入到 p 的后面

B. 把 p 指向的结点插入到 q 的后面

C. 把 p 指向的结点插入到 h 的后面

D. 把 q 指向的结点插入到 h 的后面

. 与中缀表达式 x+y*(z-u)/v 等价的后缀表达式是（ A ）。

A. xyzu-*v/+

B. xyzu-v/*+

C. x/*y-zuv

D. +x*y/-zuv

给定最大堆 28, 22, 20, 19, 8, 12, 15, 5，对该堆进行两次删除操作后，得到的新堆是（ B ）。

A. 20, 19, 15, 12, 8, 5

B. 20, 19, 15, 5, 8, 12

C. 20, 19, 12, 15, 8, 5

D. 20, 19, 8, 12, 15, 5

4. 一队同学在操场上排队，但是现在已经站好的队形是错误的，需要根据如下规则重新排队：每个同学持有一张写着一个字母的牌子，范围是从 A 到 Z，排队的流程是：每个同学依次进入栈中，每次操作可以选择让一个同学进栈或出栈，出栈序列即为最终的排队顺序。现在已知队伍中有 6 名同学，每名同学的编号依次是 B, B, C, B, B, A，正确的队形是 B, B, C, A, B, B，满足上述条件的排队方式共有（ A ）种。

A．4　　B．5　　C．6　　D．7

C

5. Lisp 是一种历史悠久的高级编程语言，由 John McCarthy 在 1958 年提出，在 Lisp 中使用 S 表达式来表示数据和代码，S 表达式可以是原子类型（单一的不可分元素，如数字、符号），也可以是其他原子类型组成的复合结构（如列表）。因此，在底层设计中常采用（ B ）这一数据结构来实现 S 表达式。

A．二叉树　　B．广义表　　C．有向图　　D．散列表

6. 使用 KMP 算法计算字符串 ababaaababaa 的 next 数组是（ D ）。

A．-1, 0, 0, 1, 2, 3, 1, 1, 1, 2, 3, 4

B．-1, 0, 0, 1, 2, 3, 4, 1, 2, 3, 4, 5

C．-1, 0, 0, 1, 2, 3, 1, 1, 2, 3, 4, 5

D．-1, 0, 1, 1, 2, 3, 1, 1, 2, 3, 4, 5

7. 现定义自然数数列 L，以 a 为基的 L 定义如下：

（1）a 是 L 的基，且 a 是 L 的第一个元素；

（2）如果 x 在 L 中，则 2x+1 和 3x+1 也都在 L 中；

（3）没有其他元素在 L 中；

当 a＝2 时，如果将 L 中元素按照升序排列，L 中第 8 个元素是（ C ）。

A．16　　B．22　　C．23　　D．25

8. 若结点 p 与 q 在二叉树的中序遍历序列中相邻，且 p 在 q 之前，则下列 p 与 q 的关系中，不可能的是（ C ）。

A. q 是 p 的双亲

B. q 是 p 的右孩子

C. q 是 p 的右兄弟

D. q 是 p 的双亲的双亲

9. 已知二维数组 A 按行优先方式存储，每个元素占用 1 个存储单元。若元素 A[0][0]的存储地址是 100，A[3][3]的存储地址是 220，则元素 A[5][5]的存储地址是（ B ）。

A. 295　　B. 300　　C. 301　　D. 306

10. 现在要求使用两个满足后入先出原则的栈 A 和 B 来实现一个满足先入先出原则的队列。现规定栈 A 用来实现入队操作，在入队时直接将元素压入栈 A 即可，栈 B 用来实现出队操作，下面对于队列入队与出队操作的实现与分析，错误的是（ ①D ）

①在元素出队时若栈 B 为空，直接将栈 A 中所有元素依次弹出压入栈 B 后弹出栈 B 栈顶元素即可；若栈 B 不为空，直接将栈 B 栈顶的元素弹出即可

②在元素出队时直接将栈 B 栈顶的元素弹出即可

③两个栈实现队列最快可以在 O(1)的时间复杂度内实现入队与出队操作

④两个栈实现队列最快可以在 O(1)的时间复杂度内实现入队操作，O(n)的时间复杂度内实现出队操作

A. ①③　　B. ①④　　C. ②③　　D. ②④

二、填空题（每空 2 分，共 20 分）

得分	20
评分人	X

1. 一棵完全二叉树上有 500 个结点，其中叶子结点的个数是 250 。

2. 广义表 L=（a，b，c，（d，（e，f，g））） 的长度是 4 ，利用 Head（简写 H）和 Tail（简写 T）运算从 L 中取出原子 d 的操作是： ~~H(H(T(T(T(L)))))~~ H(H(T(T(T(L))))) 。

3. 循环队列 Q 存储于数组中（数组空间为 K），front 指向队头元素，rear 指向队尾元素之后的空位置，队列中的元素个数是 ~~(rear-front+K) mod K~~ (rear-front+K-1) m

4. 将三对角矩阵 A 中三条对角线上的元素按行存放在一维数组 B 中，且 A[0][0]存放于 B[0]，则 A[11][12]存在 B[34]中。

5. 具有 4 个结点的不同的二叉树的棵数是 14 。

6. 以二叉链表作为二叉树存储结构，对有 N 个结点的二叉链表进行中序线索化时，最终作为线索的非空指针域的个数为 N-1 。

7. 若入栈序列为 1，2，3，4，5，6，并且 2 第一个出栈，则有 42 种不同的出栈序列。

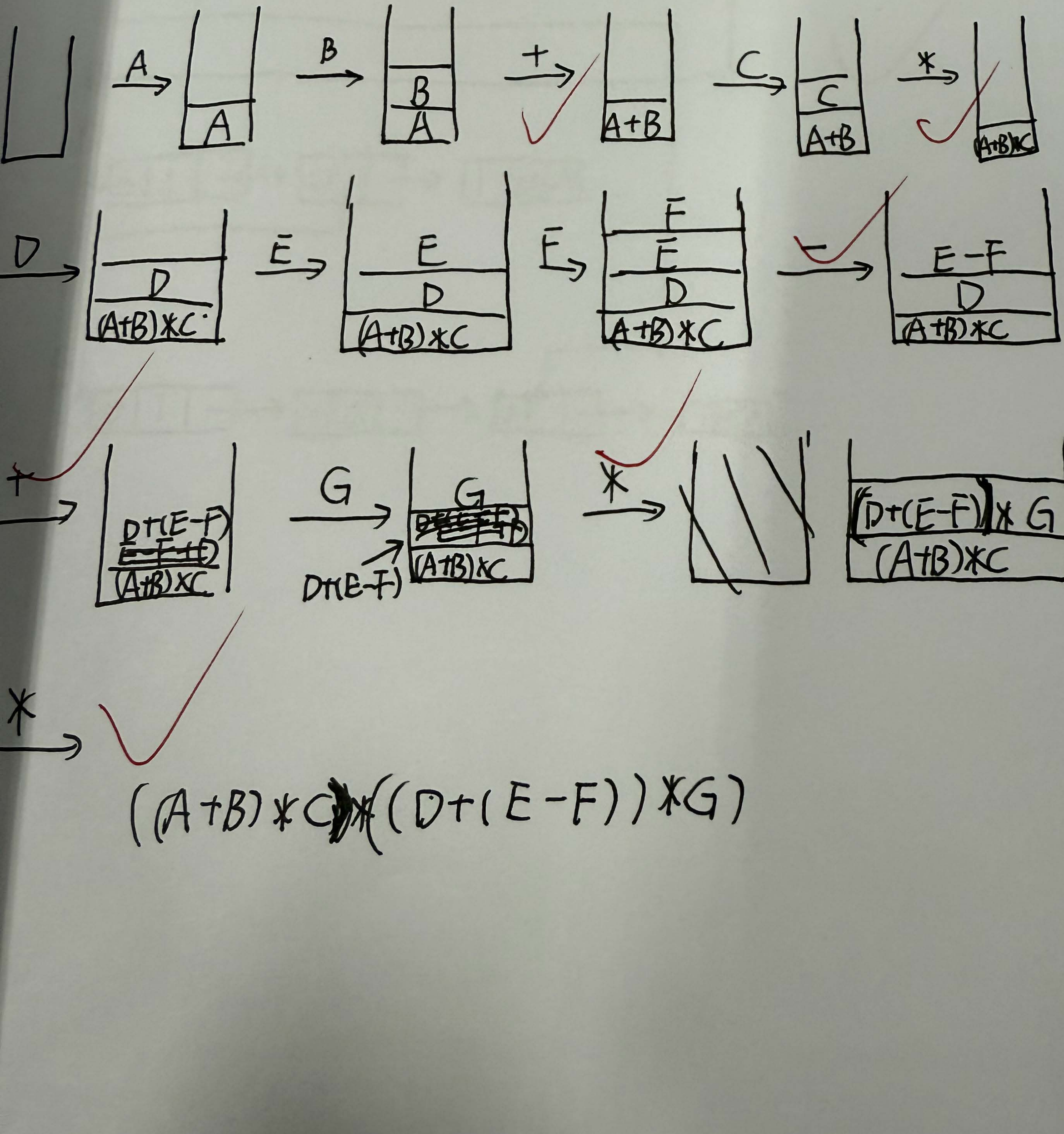
8. 通过 SiftDown 向下调整的方式构建一个包含 n 个记录的最大堆，其紧致的时间复杂度是 O(n) 。

9. 设权值集合 W={6，13，2，7，32，3，21，10}，根据这些权值构造哈夫曼树，则这棵哈夫曼树的高度为 6 。（注：如果某棵树只有根结点，则该树的高度为 1）

评分人	

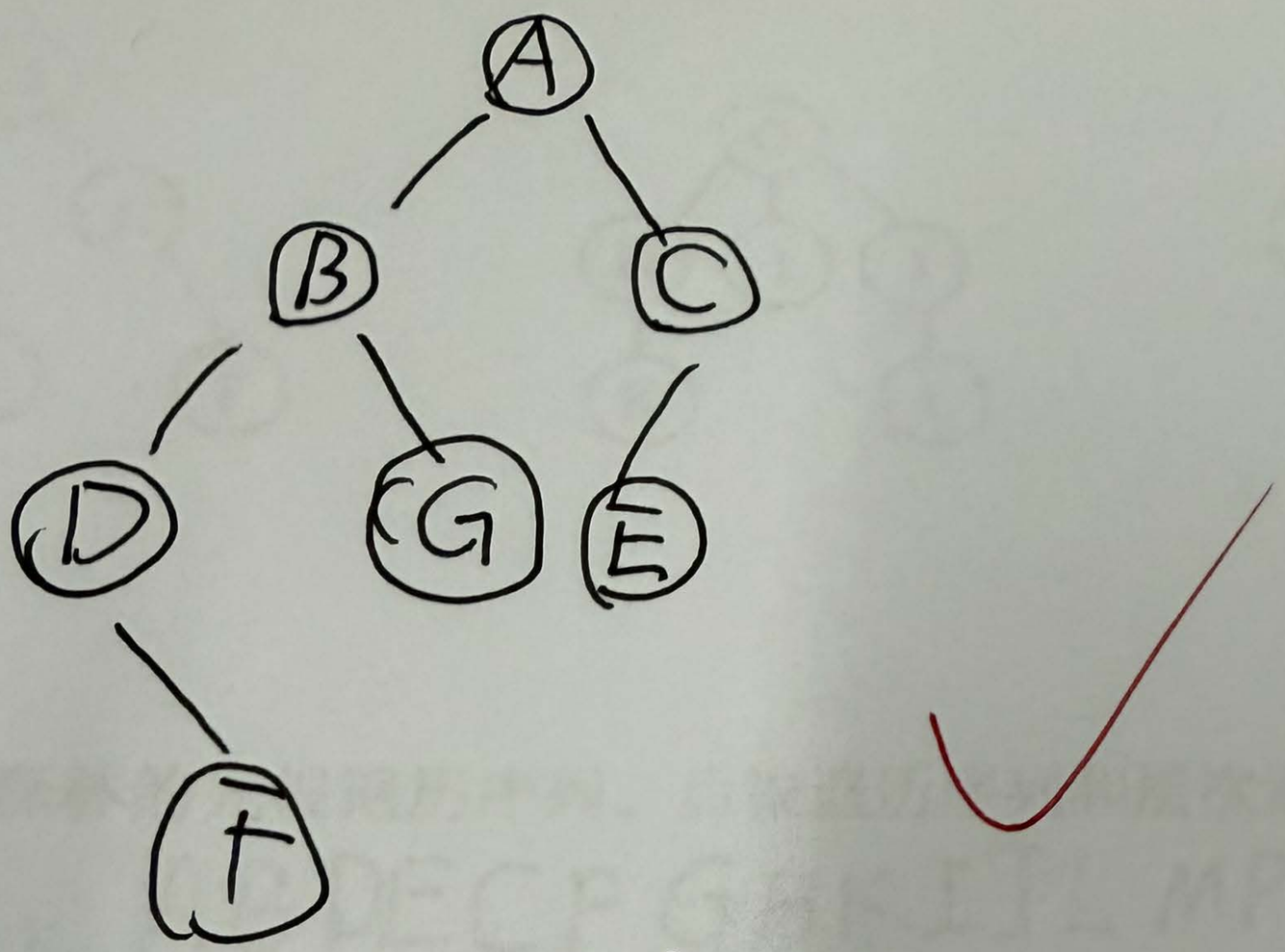
1. 借助栈可以实现后缀表达式 A B + C * D E F - + G * * 的计算，写出该计算中栈的详细变化过程。

输入	栈内元素	关系	操作
A	空	\	输出A



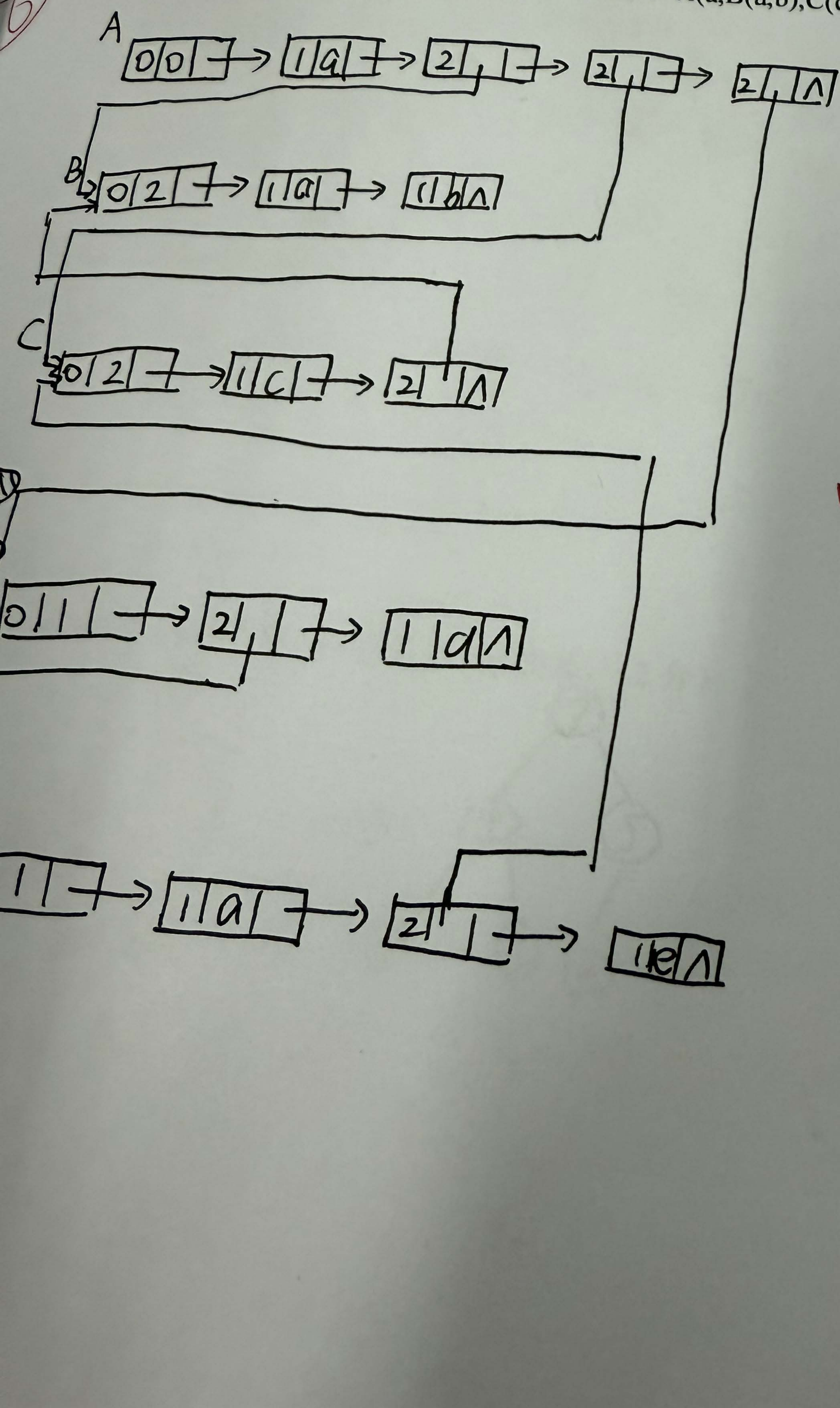
((A+B)*C)*(D+(E-F))*G)

2. 假设一棵二叉树的中序遍历序列为 DFBGAEC，后序遍历序列为 FDGBECA，请画出该二叉树，并计算该二叉树的前序遍历序列。

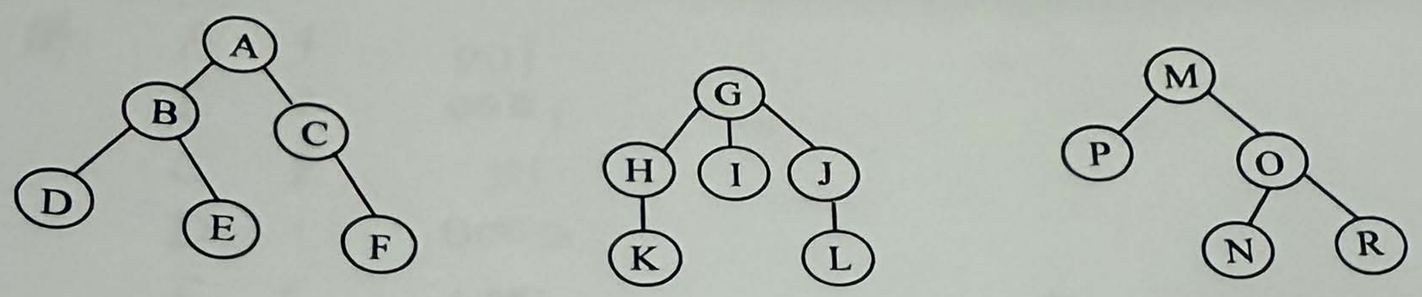


前序 ABDFGCE

3. 以链表结构的形式画出下列广义表的存储表示：A(a,B(a,b),C(c,B),D(E(a,C,e),d)) 。



4. 对于下图所示的森林：



(1) 求森林的先根遍历序列、后根遍历序列和层次遍历序列； (3 分)

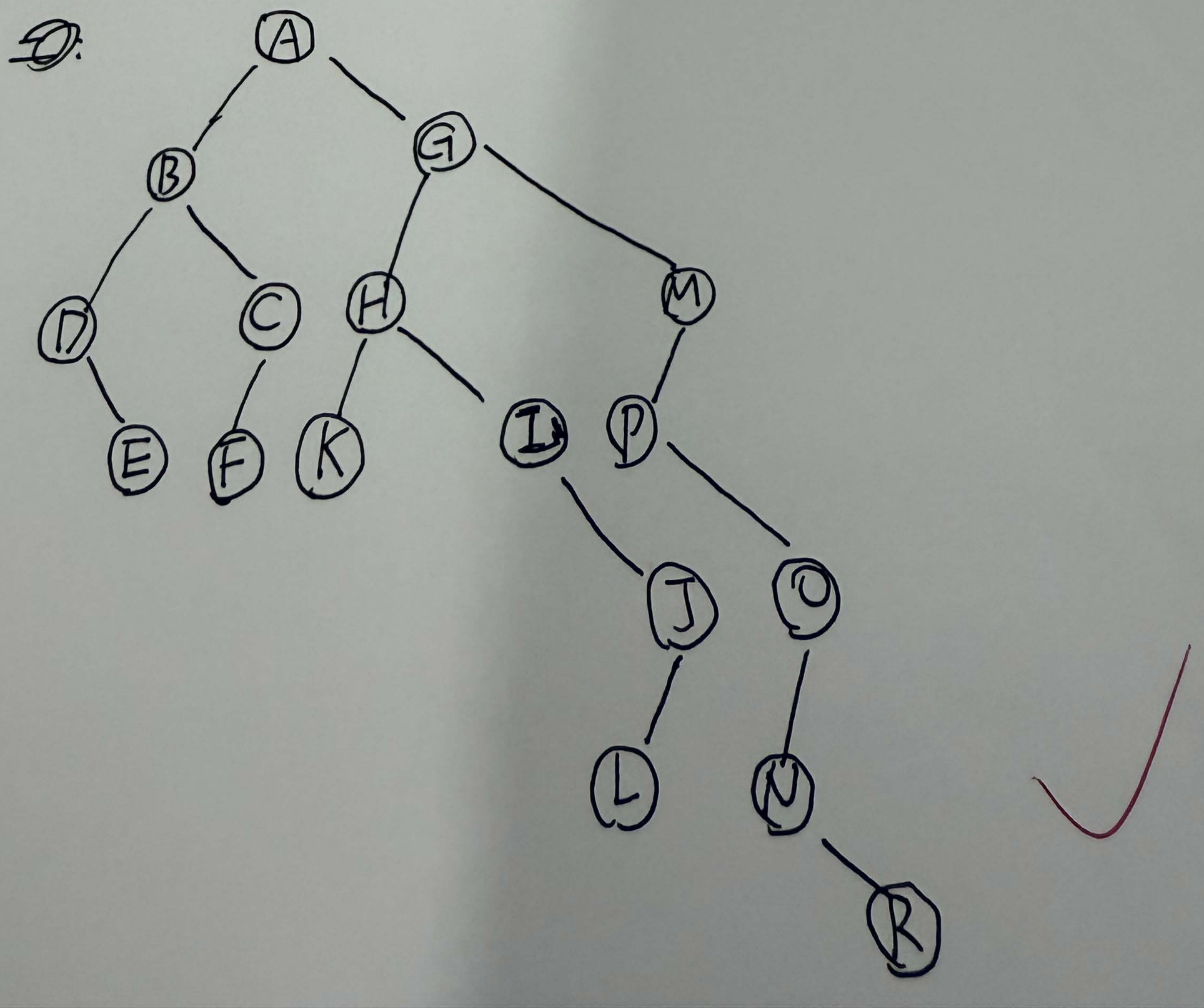
先：ABDECFGHKIJLMPONR

后：DEDFCBKHLJIHRNOPMGA

层：AGMBCHIJPODEFKLNR

DEBFCA KHILJG PNROM

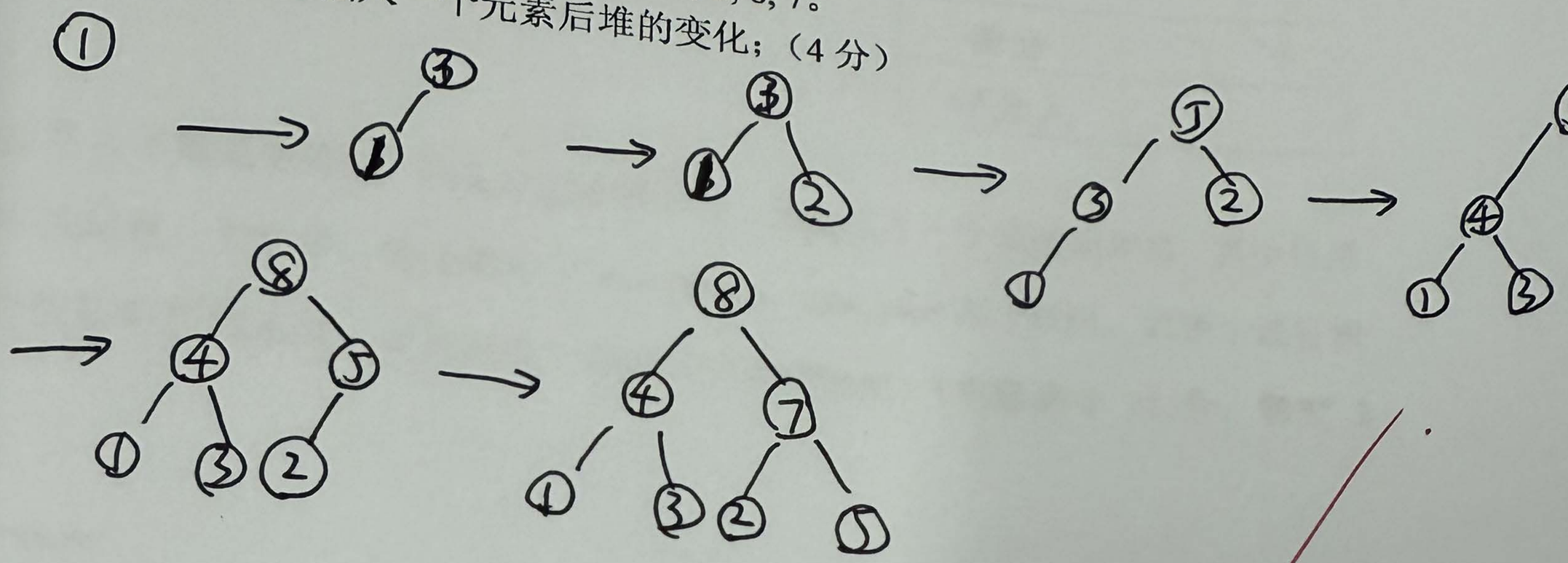
(2) 将此森林转换为相应的二叉树表示。(3 分)



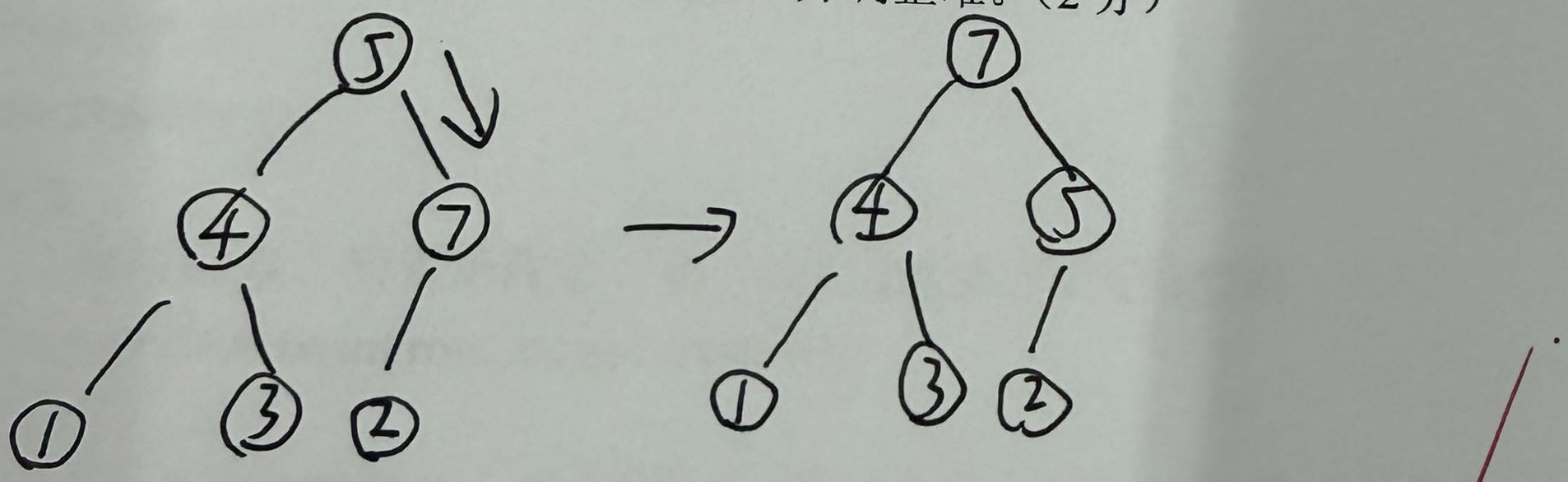
+6

5. 最大堆初始为空，然后依次插入元素 1, 3, 2, 5, 4, 8, 7。

(1) 以树的形式画出每插入一个元素后堆的变化；（4 分）



(2) 从构造好的堆中删除最大元素，并调整堆。（2 分）

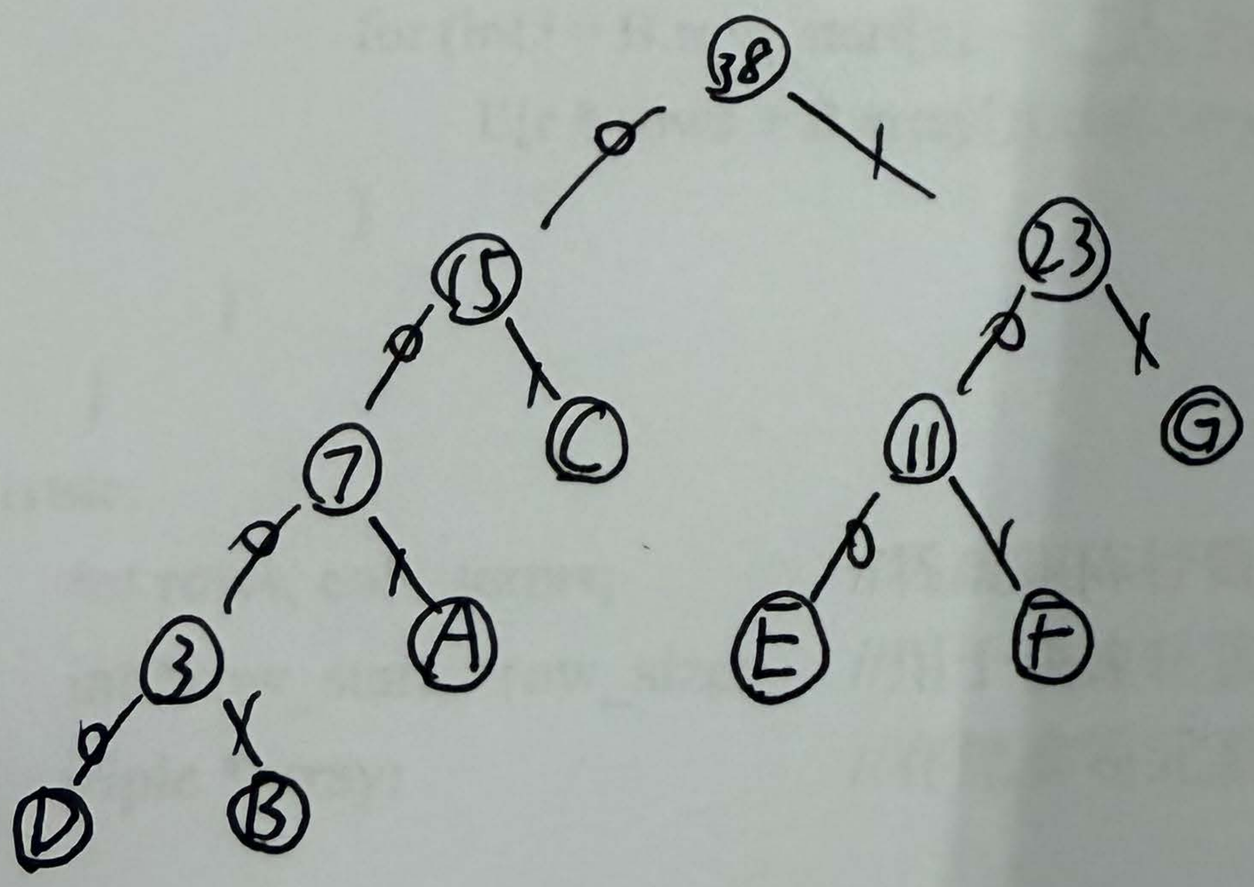


+6

6. 假设用于通信的电文仅由{A, B, C, D, E, F, G} 7 个字母组成，这些字母在电文中出现的频率分别为 4，2，8，1，5，6，12，请计算每个字母的哈夫曼编码，写出详细的编码计算过程。

解：

A	4	001
B	2	0001
C	8	01
D	1	0000
E	5	100
F	6	101
G	12	11



得分	24
评分人	

1. 小蓝鲸在学习了稀疏矩阵后，决定自己实践一下。他定义了一个稀疏矩阵类，其中包括稀疏矩阵的转置、乘法等，他还维护了 **row_start**、**row_size** 两个数组，以便于进行快速转置并优化乘法的速度。请你帮他一起完成代码实现吧！（**本题满分 12 分，每空 2 分**）

```
#include <iostream>
using namespace std;
//定义三元组
struct triple {
    int row, col, value;
};
//定义稀疏矩阵类
class sparseMatrix {
    public:
        //构造函数，设置矩阵行、列、非零元素数并创建各数组
        sparseMatrix(int row, int col, int term);

        //向 array 数组的第 i 个位置插入非零元素并设置三元组各变量的值
        void Insert(int row, int col, int value, int i);

        //根据稀疏矩阵当前存储元素更新 row_start、row_size 数组
        void Trim();

        //稀疏矩阵的快速转置
        sparseMatrix Transpose() {
            //构造转置矩阵 A
            sparseMatrix A(cols, rows, terms);
            if (terms > 0) {
                //初始化并计算转置后的 A.row_size 数组
                for (int i = 0; i < cols; i++)
                    A.row_size[i] = 0;
                for (int i = 0; i < terms; i++)
                    A.row_size[ A.array[i].col ]++; // 第(1)空
                //计算转置后的 A.row_start 数组
                A.row_start[0] = 0;
                for (int i = 1; i < cols; i++)
                    A.row_start[i] = A.row_start[i-1]+A.row_size[i-1]; //第(2)空
                //使用 A.row_start 数组向转置矩阵 A 中存入非零元素
```

12

```
        for (int i = 0; i < terms; i++) {
            int j = A.row_start[array[i].col];   //第(3)空
            //向 A 的稀疏矩阵数组第 j 个位置插入转置后的元素
            A.Insert(array[i].col, array[i].row, array[i].value, j);
            A.row_start[array[i].col]++; // 第(4)空
        }
        //恢复 A.row_start 为正确的值
        A.Trim();
    }
    return A;
}
//稀疏矩阵乘法，矩阵与 B 相乘，运算结果输出到一维数组 E（初始全为 0）中
void Multiply(sparseMatrix& B, int* E) {
    for (int i = 0; i < terms; i++) {
        int r = array[i].row, c = array[i].col;   // 第(5)空
        for (int j = B.row_start[c]; j < B.row_start[c]+B.row_size[c]; j++){//第(6)空
            E[r * rows + B.array[j].col] += array[i].value * B.array[j].value;
        }
    }
}
private:
    int rows, cols, terms;          //稀疏矩阵行数、列数、非零元素数
    int *row_start, *row_size;      //用于快速转置和优化乘法的辅助数组
    triple *array;                  //存储非零元素的数组
};
```

2. 在小蓝鲸王国里，小蓝鲸王准备进行一场家族树的探索任务。国王希望小蓝鲸不仅熟悉每一位家族成员的关系，还能理解他们之间的亲密度。因此，国王设立了一个挑战：将家族树中每个成员的“亲密值”进行替换，使小蓝鲸能直观地感受到亲缘关系的力量。

注意：如果两个结点的父结点相同，那么它们互为**兄弟**结点；如果两个结点的父结点是**兄弟**结点，那么它们互为**堂兄弟**结点。

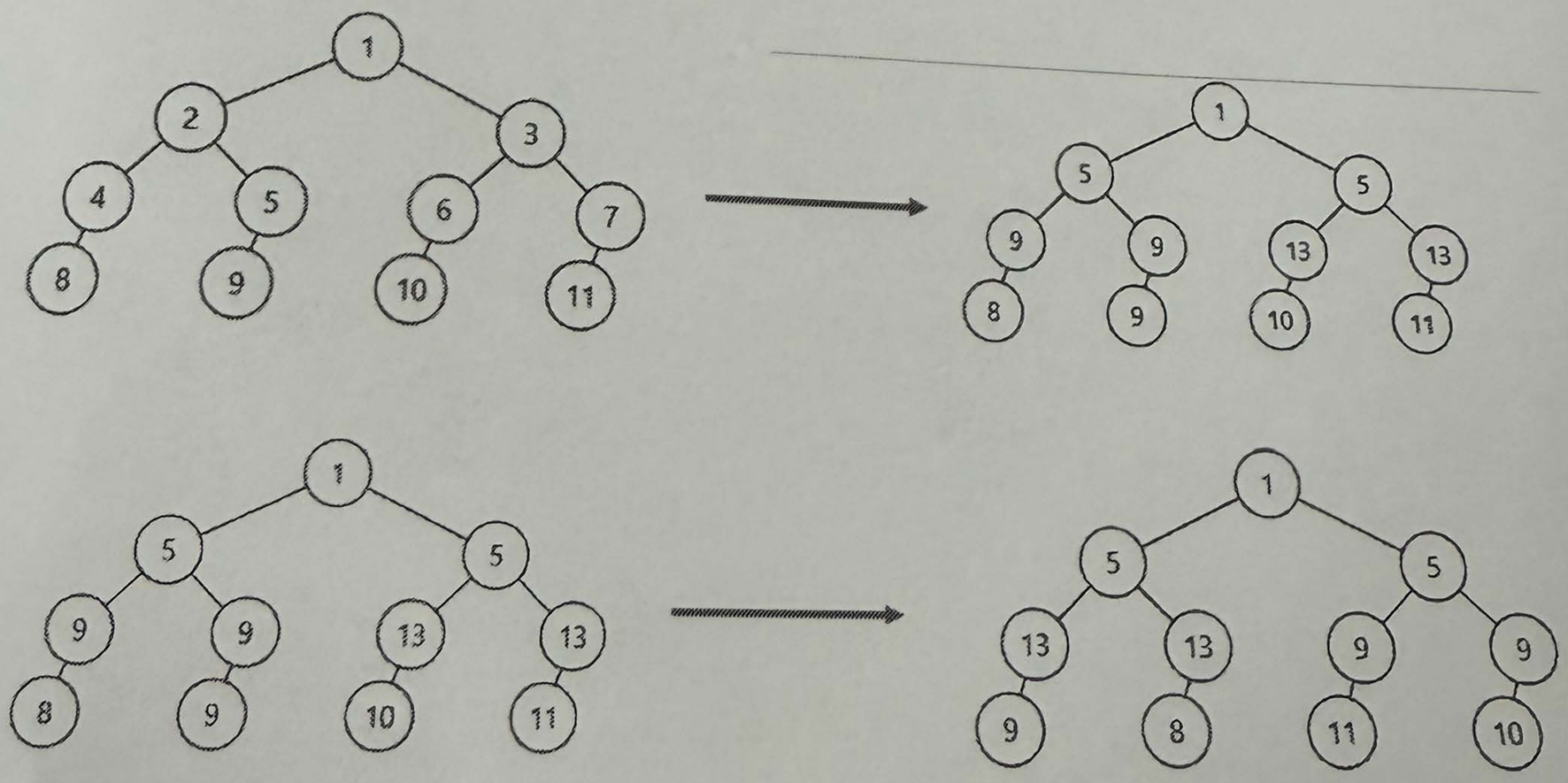
（1）小蓝鲸需要将家族成员的“亲密值”替换成他和他的**兄弟**的亲密值之和。请先写出算法思路，并写出 C++程序伪代码（思路 3 分，程序 3 分）；

void Q1(TreeNode* root) //给定一棵二叉树的根 root，请你帮助小蓝鲸将每个结点的值替换成该结点及其**兄弟结点**的结点值之和

（2）小蓝鲸需要找到每个成员的堂兄弟，即父辈们是兄弟的成员，并将每个成员的“亲密值”替换为其**堂兄弟**们的亲密值之和。请先写出算法思路，并写出 C++程序伪代码（思路 3 分，程序 3 分）。

void Q2(TreeNode* root) //给定一棵二叉树的根 root，请你帮助小蓝鲸将每个结点的值替换成该结点的***所有堂兄弟结点***值的和（如果不存在堂兄弟结点则不进行替换）。

下图通过一个示例展示了从一个原始的家族亲密度二叉树开始，依次计算并替换各结点值为该结点与其兄弟的亲密值（第一小题），并利用替换结点后的二叉树，进一步将各结点值替换成为其堂兄弟们的亲密值之和（第二小题）。




```
struct TreeNode {
    int val;//该结点的值
    TreeNode* left;
    TreeNode* right;
    int BrotherSum; // 该结点与其兄弟之和.
};//可以在结点类中增加元素，但是都需要自己进行计算
```